

Unit - I Introduction

- 1.1 Introduction to Python
- 1.2 Features of python
- 1.3 What Can I Do with Python?
- 1.4 Python Interpreter
- 1.5 Data types, Variables, Comments, Operators, expressions; input, processing and output statements
- 1.6 Control Structures: loops and decision

Unit – II String Handling, Classes, Modules and Package

- 2.1 Strings, String operations and String Slicing
- 2.2 Defining Classes
- 2.3 Defining and calling functions passing arguments to functions
- 2.4 Python and OOP – Inheritance, polymorphism
- 2.5 Modules – datetime, math
- 2.6 Packages

Unit – III - Exception Handling and Collections

- 3.1 Exception in python
- 3.2 Exception roles
- 3.3 Exception Handling
- 3.4 Collections in Python – List, Tuples, Dictionaries, Sets

Unit – IV - GUI Programming and Database Connectivity Using Python

- 4.1 Graphical User Interfaces
- 4.2 Using the tkinter Module
- 4.3 Creating Label, Text, Button, info Dialog Boxes, Radiobutton, Checkbutton
- 4.4 Getting Input
- 4.5 Importing MySQL for Python
- 4.6 Connecting with a database
- 4.7 Forming a query in MySQL
- 4.8 Passing a query to MySQL

1.1 Introduction to Python

- **Python** is a general-purpose interpreted, interactive, object-oriented, and high-level programming language.
- It was created by **Guido van Rossum during 1985- 1990.**
- Like Perl, Python source code is also available under the GNU General Public License (GPL).
- Python is designed as highly readable language as it uses many english keywords frequently.
- It uses PVM (Python Virtual Machine) to translates python script into bytecodes.
- Thus python code becomes platform and operating system independent
- Python also supports RAD on different OS platforms like MacOS, Windows, Unix, etc as well as python can also be executed on MS-DOS, OS/2 and other operating systems.

[python name taken from Comedy group named Monty Python, this group was having many popular comedians as members].

- Python as prog language shows following imp characteristics:-
 - It supports functional and structured programming methods.
 - It supports OOP programming methods.
 - It can be used as scripting language to build large applications.
 - Supports high-level dynamic data types and supports dynamic type checking.
 - It supports automatic garbage collection.

Unit - I Introduction

- It can be easily integrated with other languages like C, C++, COM, ActiveX, java, etc
- Python provides many **benefits** like-
 - Easy to learn – as it uses few keywords, simple structure and simple syntax.
 - Easy to read – as it uses many English keywords hence easy to read
 - Easy to maintain – due to structured programming, python code is easy to maintain.
 - Collection of std library – to performs different operations it provides many std lib to make the code easy.
 - Interactive mode - supports interactive way of testing and debugging of python code.
 - Portable – it can run on any hardware or OS platform.
 - Extendable – new modules can be easily added to existing code so that it can be extended.
 - Database Interaction – Python can be connected with any std database application to read and write data.
 - GUI Programming – Graphics interfaces can also be created using python.
 - Scalable – python code size can be changed at any time and can also handle small as well as large size data.
- Major historical development in Python language are:-

- Python 1.5.2 – Released in April-1999. It includes most the functionalities required in Modern Prog language.
- Python 1.6. – Released in Sep-2000. In this new features like list objects, socket and string number conversion tools are included.
- Python 2.0 – Oct-2000 – in this old Java-based python interpreter is renewed to hybrid interpreter ie. Jython hybrid.
- Python 3000 – still under development as Python 3000 code name.

1.2 Features of python

- 1. Python is Free** – Python can be implemented without purchase of any software or there is no licensing issues.
- 2. Portable** – python code can be developed on any hardware or software platform and can be easily transferred to any other hardware or software platform.
- 3. Extensible** – The complete python language is written in C language and it's original code is also available, hence any one can download it, and can add new features to it, thus python language can be extended to support new features.
- 4. Database Interaction** – Python can be connected with any std database application to read and write data.
- 5. GUI Programming** – Graphics interfaces can also be created using python.
- 6. Scalable** - python code size can be changed at any time and can also handle small as well as large size data

7. C/C++ Embedding – Python code can be embedded into C or C++ applications.

8. Powerful language - python library comes with huge library of useful modules that supports programmer to perform

- Network communication
- Text processing
- Regular expression matching
- Low-level operations of Operating system

- The other features of python can be enlisted as:

- Easy to learn – as it uses few keywords, simple structure and simple syntax.
- Easy to read – as it uses many English keywords hence easy to read
- Easy to maintain – due to structured programming, python code is easy to maintain.
- Collection of std library – to performs different operations it provides many std lib to make the code easy.
- Interactive mode - supports interactive way of testing and debugging of python code.
- Portable – it can run on any hardware or OS platform.
- Extendable – new modules can be easily added to existing code so that it can be extended.
- Database Interaction – Python can be connected with any std database application to read and write data.
- GUI Programming – Graphics interfaces can also be created using python.

- Scalable – python code size can be changed at any time and can also handle small as well as large size data.

1.3 What Can I Do with Python?

- As python is high-level interactive OOP language, it helps to develop application for different areas of Computer Applications like:

1. **Mathematics** - To develop scientific and engineering applications many mathematical expressions are required in simplified form
 - Python provides extensions/library i.e NumPy which allows to interact and use mathematics libraries.
 - It helps to performs simple calculations as well as highly-complicated calculations with value of any digits.
2. **Text Processing** –
 - Python provides all the functionalities/modules required for text processing like
 - Splitting
 - Separating &
 - Summarizing of data.
 - With the help of data types, data collected can be stored for processing.
 - With regular expression, data validations can be performed like checking of valid email id, contact number, etc.
 - Also python allows to give output in different formats like HTML, GIF, etc
3. **RAD** – python provides many built-in libraries hence it allows to develop applications at high speed. Due to interpreter facility, every time,

after making changes no time to spend on compilation, directly it can be interpreted.

Also, due to interactive de-bugging facility, it is more easy and fast to find error and solve it.

4. Cross-Platform Development – As python is platform independent (portable) language, hence it's code can be executed on any hardware and software platform.

This is very important when in network same python application to be executed and in such network systems are using different hardware and OS platform.

5. System Utilities – python can is allowed to access low-level functionalities of OS hence a powerful python code can be developed to make system utility such as Operating system tools e.g. Antivirus, Disk Cleanup Tool, Backup & Recovery Tools etc

6. Internet Programming – python provides powerful modules that allow to communicate using **network sockets** at both levels :

- a. Basic level
- b. Protocol level
- Python code can read email using POP server
- Can give input to web-server and get high-quality formatted output

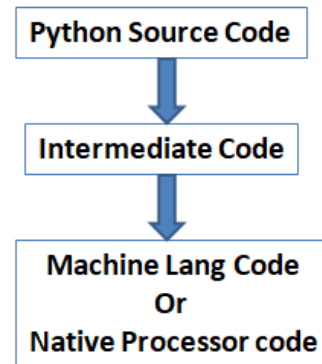
7. Database Programming – Python provides many modules that helps to connect python code to be connected with different database like Oracle, MYSQL, Informix, MS-SQL etc.

- As python is high level interpreted language, it's execution requires Python interpreter.

- An **interpreter** is a kind of program that executes other programs.

- Python code first converted into intermediate code i.e. byte code (as in java language).

- This intermediate code is translated in machine language code for execution.



- Once python code is converted into byte code, it is saved in a file with .pyc extension.

- Python interpreter takes raw-text source code and executes each statement.

- Python statements can be executed in following ways-

- Directly python statement can be given to interpreter – any executable statement can be given to python interpreter prompt >>> as

e.g. >>> print ("Hello")

or

>>> print ('Hello')

- Python statement can be saved in file and then executed by interpreter.

Start Python Idle → go to file menu →

New → type the python code → File menu

→ Save → give filename

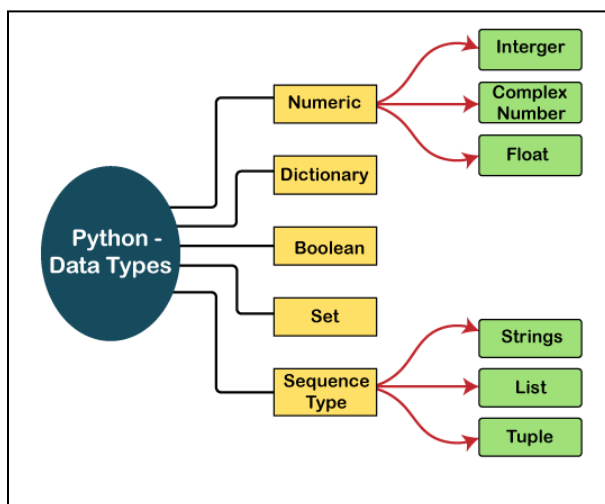
1.4 Python Interpreter

In Run menu → Select Run Module or
Press F5

e.g.

1.5 Data types

- Python as programming language supports data types using which variable can be created.
- Python provides following standard data types which decides How which value to be stored?



1) **Numeric** – It represents numeric value in following different forms-

a. **Integer** – numeric value without decimal point.

e.g. `a=100`

b. **Complex number** – it combination multiple real or floating numbers. In this, the number must be suffixed by j or J as

`a=1.3+3.4j`

c. **Float** – it is a single numeric value with decimal point as

e.g. `a=12.34`

2. **Dictionary** – It is an associative array i.e. collection of unordered set of data elements.

Each data element is also must be associated with key.

Multiple value-key pair must be separated by comma and all values must be enclosed in curly braces as:

`<DictName>={Key:Value,Key:value,.....}`

e.g. `values={1:100,5:250,3:120}`

- in same dictionary, key can be of same type or different types.

e.g. `values={1:'Abc',5.5:250,'X':12.90}`

- In this, values can be duplicate, but keys must be unique

- if key is repeated, then previous value of same key is replaced by new value.

- Dictionary supports following useful methods

i) `keys()` – returns only the keys used to create dictionary items.

ii) `values()` – returns only the values in dictionary.

iii) `clear()` – deletes all the items in the dictionary.

`-len()` function can be used to count number of items present in dictionary as:

`print(len(values.keys()))`

e.g

`a={1.8:200,7:'Abc','X':45.7}`

`print("All Keys and values=",a)`

`print("Total Number of Items",len(a.keys()))`

`print("All Keys are=",a.keys())`

`print("All values are=",a.values())`

```
All Keys and values= {1.8: 200, 7: 'Abc', 'X': 45.7}
Total Number of Items 3
All Keys are= dict_keys([1.8, 7, 'X'])
All values are= dict_values([200, 'Abc', 45.7])
... |
```

3. **Boolean** – This data type provides two built-in values True and False. It can be used as:

```
a=True
```

```
print(type(a))
```

note: T and F of values must be capitals, if used small, will show error.

4. Set

- Set datatype in python represents the unordered collection of the data type.

- It is iterable, mutable (can modify after creation), and has unique elements.

- In set, the order of the elements is undefined; it may return the changed sequence of the element.

- values in set can be of same or different types:

- The set can be created in one of the following ways:

- i) by using a built-in function **set()**, as:

```
setName=set((value1,value2,value3.....))
```

```
a=set(('Abc',300,35.77))
```

```
print("All the Elements in Set=",a)
```

output-

```
All the Elements in Set= {35.77, 300, 'Abc'}
```

- ii) group of elements is passed in the curly braces and separated by the comma. It can contain various types of values.

```
a={44,'Abc',89.09,"Xyz",100}
```

```
print("All the Elements in Set=",a)
```

output-

```
All the Elements in Set= {100, 'Abc', 44, 89.09, 'Xyz'}
```

3. **Sequence Type** – Sequence data type represents collection of data elements. Python supports following sequence sub-types

- i) **Strings** – It represents sequence of character or collection of bytes in quotation marks.

- Python allows, string to be enclosed in single, double or triple quotes.

e.g.

```
s1='FYBCA'
```

```
s2="SYBCA"
```

```
s3="''TYBCA''"
```

```
print("String-1=",s1)
```

```
print("String-2=",s2)
```

```
print("String-3=",s3)
```

- As like java & other languages, string indexing in python also starts with zero.

- This indexing helps in performing different string manipulations like

- a) **Extracting single char** – a single character at given position can be extracted as:

```
StringName[Index]
```

e.g.

```
s1='Hello India'
```

```
print("Sub String=",s1[3])
```

- b) **Extracting sub-string** – to extract set of characters, start index and number of characters to be specified as:

```
StringName[startIndex:NumberofChars]
```

```
s1='Hello India'
```

```
print("Sub String=",s1[0:3])
```

- c) **Repeating string** – same string can be repeated using * operator as

```
StringName * int
```

Eg

```
s1='Hello'
```

```
print("Repeated String=",s1*3)
```

Output –

```
Repeated String= HelloHelloHello
```

ii) **List** – It represents “list” class type of python language.

- Python list is similar to array of C, C++ and java like languages i.e. to store collection of data elements.
- But it also can contain multiple elements of same as well as different data types.
- To store data items in list, items to be separated by comma and enclosed in brackets []

e.g.

```
data=["Abc",1001,78.99,'A']
```

```
print ("Current Data=",data)
```

output

```
Current Data= ['Abc', 1001, 78.99, 'A']
```

- On list following different **operations** are supported:

- o **: (Slicing)** - it is used to slice or separate list data items as:

`print(data1[2:])` – it will print all from elements from 2nd index as

```
[78.99, 'A']
```

`print(data1[1:3])` – it will print all from elements from index 1 to 3-1

```
[1001, 78.99]
```

- o **+ (Concatenation)** - it is used to concatenate items of multiple list as:

```
data1=["Abc",1001,1.11,'Z']
```

```
data2=["Xyz",2001,2.22,'Z']
```

```
print (data1+data2)
```

output –

```
['Abc', 1001, 1.11, 'Z', 'Xyz', 2001, 2.22, 'Z']
```

- o ***(Repetition)** - to repeat each data items in list for given number of times:

```
data1=["Abc",1001,1.11,'Z']
```

```
print (data1*2)
```

output

```
['Abc', 1001, 1.11, 'Z', 'Abc', 1001, 1.11, 'Z']
```

- List is dynamic data structure, hence allows to add new elements as well as removes existing elements from list at runtime
- For this, it supports following methods

- o **append(DataItem)** – it add new element at the end of existing list as

e.g. `data1.append(100)`

- o **insert(index,value)** – it inserts new element at the given index by shifting existing element forward

e.g. `data1.insert(1,'Demo')`

- o **remove()** - it removes existing data element from list with backward shift.

e.g. `data1.remove(1001)`

if given value/item not exiting, then generates error - `ValueError: list.remove(x): x not in list`

e.g.

```
data1=["Abc",1001,1.11,'Z']
```

```
print ("Current Data Elements=",data1)
```

```
data1.append(100)
```

```
print ("After Appending=",data1)
```

```
data1.insert(1,'DEMO')
```

```
print ("After Insert",data1)
```

```
data1.remove(1001)
```

```
print ("AFTER removing",data1)
```

output

```
Current Data Elements= ['Abc', 1001, 1.11, 'Z']
```

```
After Appending= ['Abc', 1001, 1.11, 'Z', 100]
```

After Insert ['Abc', 'DEMO', 1001, 1.11, 'Z', 100]

After removing ['Abc', 'DEMO', 1.11, 'Z', 100]

iii) Tuple – It is one sequence type in Python language and it represents “tuple” lib class of java lang.

- It represents collection of data items of same as well as different types.
- It is alike list, but with imp difference that – **it is read-only data structure i.e. the data items in list can not be added or removed at run-time**
- The items of the tuple are separated with a comma (,) and enclosed in parentheses ().

e.g.

```
data1=("Abc",1001,1.11,'Z')
```

```
print ("Current Tuple Values=",data1)
```

output –

```
Current Tuple Values= ('Abc', 1001, 1.11, 'Z')
```

- As like list, Tuple also supports following operations
 - **: (Slicing)** - it is used to slice or separate list data items as:

```
print(data1[2:])
```

 – it will print all from elements from 2nd index as

[1.11, 'Z']

```
print(data1[1:3])
```

 – it will print all from elements from index 1 to 3-1

[1001, 1.11]

- **+ (Concatenation)** - it is used to concatenate items of multiple list as:

```
data1=("Abc",1001,1.11,'Z')
```

```
data2=("Xyz",2001,2.22,'Z')
```

```
print (data1+data2)
```

output –

```
['Abc', 1001, 1.11, 'Z', 'Xyz', 2001, 2.22, 'Z']
```

- ***(Repetition)** - to repeat each data items in list for given number of times:

```
data1=("Abc",1001,1.11,'Z')
```

```
print (data1*2)
```

output

```
['Abc', 1001, 1.11, 'Z', 'Abc', 1001, 1.11, 'Z']
```

***Variables**

- Python program may needs data element for it's execution.
- Such data element can be stored using variable so that it can be processed any time.
- Variable is reserved memory location to hold/store required value.
- For which variable, how much memory is required is decided on the basis of data type of that variable.
- as per the datatype or type of value stored in variable, interpreter decides and allocates memory to variable.
- A python prog may uses single or multiple variables.
- As oppose to C,C++,Java languages, **Python does not needs explicit declaration** for variable & to reserve memory space.
- Based on the data type of a variable, Therefore, by assigning different data types to variables, you can store integers, decimals or characters in these variables.
- While creating variable, a valid variable name must be assigned as per variable naming conventions.
- Value to any variable can be assigned in two ways:

1) Static assignment 2) Dynamic assignment

1) Static assignment – in this, value to any variable can be assigned directly in prog code using = operator as:

VariableName=value

e.g. ctr=100

Prog code-

```
a=12
b=15.67
c='A'
d='Abc Xyz'
```

```
print("Type of variable a=",type(a),"value in
a=",a)
print("Type of variable a=",type(b),"value in
a=",b)
print("Type of variable a=",type(c),"value in
a=",c)
print("Type of variable a=",type(d),"value in
a=",d)
```

Output –

```
('Type of variable a=', <class 'int'>,
'value in a=', 12)
('Type of variable a=', < class 'float'>,
'value in a=', 15.67)
('Type of variable a=', < class 'str'>,
'value in a=', 'A')
('Type of variable a=', < class 'str'>,
'value in a=', 'Abc Xyz')
```

2) Dynamic assignment – value to variable can be assigned dynamically at runtime using input() method as:

```
VariableName=input("Dummy Msg")
num=input("Enter a number=")
```

- Based on input value, datatype of variable will be decided.
- Only one input is possible using one input() statement, hence prog can use any number of input statements.

e.g.

```
a=input("Enter a value==")
```

```
print("Type of variable a=",type(a),"value in
a=",a)
a=input("Enter a value==")
print("Type of variable a=",type(a),"value in
a=",a)
a=input("Enter a value==")
print("Type of variable a=",type(a),"value in
a=",a)
```

Output-

Enter a value=="Abc"

```
('Type of variable a=', < class 'str'>, 'value in a=',
'Abc')
```

Enter a value==35.7

```
('Type of variable a=', < class 'float'>,
'value in a=', 35.7)
```

Enter a value=='A'

```
('Type of variable a=', < class 'str'>,
'value in a=', 'A')
```

Enter a value==12

```
('Type of variable a=', < class 'int'>,
'value in a=', 12)
```

Comments- Prog may contain simple as well as complex code to complete given task.

- Also in many cases, it is possible that, code created by one programmer is to be modified by other programmer.
- In such case, the prog code must be easy to understand for any programmer.
- For this, it is recommended that, in prog at required places comment is used.

- Comment is one **read-only** statement which is neither compiled nor executed.
- It helps to enhance program readability for any programmer.
- In C,C++,Java like languages, comment can be either
 - o single line using //
 - o or multi-line using /**/
- But python supports only single line comment using # symbol.
- Any statement in python if pre-fixed with # symbol will become comment which is neither compiled nor executed as:

```
# statement
e.g.
#python uses variable
# python variable demo
A=10
print("Value in A="+A)
```

*** Operators**

- It is one type of token used by any programming language.
- ***Operator is one special character or symbol associated with pre-defined operation and when used performs it's pre-defined operations.***
- Operator to complete it's operation always uses operand which any value or variable.
- Operators as one of the basic/fundamental entity of programming language helps to apply required logic in prog code.
- python supports following different types of operators –

- 1) **Arithmetic operators** – these are used to perform arithmetic/mathematical

operations on given operands. These operators are like:

+ → addition - → subtraction
 * → Multiplication / → Division
 % → modulo / remainder
 ** → Exponent
 // → floor division

- 2) **Relational or Comparison** – These operators are used to check the relation between two operands by comparison.

- o These operators returns Boolean value true or false after comparison. These operators are:

== → equality != → Not Equal
 <= → Less than or equal
 >= → Greater than or equal
 < → less than > → Greater than

These operators are mostly used to compose conditional statements in prog code.

- 3) **Assignment** – these operators are used to assign right-hand side value to left-hand side operand as

= → simple assignment += →

Addition/increase and then assignment

-= → subtraction/decrease then assignment

*= → multiply & then assignment

/= → Division & then assignment

%= → remainder assignment

**= → Exponent and then assignment

//= → Floor Division

e.g.

- 4) **Bitwise** – these operators are used to operate on low level or binary values.

5) Membership – These operators help to find if a given data element is present or is a member of a given sequence. Python supports the following membership operators:-

i) IN – It returns **TRUE** if a given entity is present in a sequence else **FALSE**.

ii) NOT IN - It returns **FALSE** if a given entity is present in a sequence else **TRUE**.

6) Identity operators – These operators are used to compare:

- i. Memory locations &
- ii. Contents of memory locations.

- Under this category, two operators are supported as:

i) **is** – returns **true** if both variables point to the same memory location else returns **false**.

ii) **is not** - returns **false** if both variables point to the same memory location else returns **true**.

Suppose there are two variables with values like

a=10

b=10

If we check these two as :

a is b → it will return true

a is not b → it will return false

- Operators help to compose an expression in program code like:

○ A=10+20

- All operators also have their pre-defined precedence and always follow the precedence rule like

○ When + (Addition) and * (Multiplication) operators are used in the same expression then, * will be evaluated first & then + like

○ A=10+5*3

▪ Value in A variable will be 25.

- Operator precedence can be changed using parenthesis () as:

○ A=(10+5)*3

▪ In the above case first, + will be evaluated & then *, hence the value in A variable will be 45.

*** Expressions**

- It is one of the common programming entities used in program code of any programming language.

- It is a program code line / statement in which operators, operands or both are used properly to make a meaningful statement.

- Python supports the following different types of expressions:-

i. Constant Expression – expression which is using a constant value e.g. a=90

ii. Arithmetic Expressions – expression using any or all arithmetic operators like

a=10+20

a=10*5+3

iii. Integral expression – expression that generates / produces only integer value e.g. a=10+20

iv. Floating expression - expression that generates / produces only floating value e.g. a=10/3 returns 3.33

v. Relational Expression – expression which is composed using relational operators (<, >, <=, etc)

v. Logical Expression - expression which is composed using logical operators (&&, or, not, etc)

vi. bitwise expression - expression which is composed using bitwise(<<<, >>>) operators.

vi. combinational expression – when in a single expression, other types of expressions are combined, it makes a combinational expression as

$a=10+(5/3)$ - combination of constant expression, arithmetic and floating expressions.

input Statements – the programming statement which is useful to get input from user during prog execution time is called input statement.

- For this python supports input() function as:
VariableName=input(“Message”)
Sname=input(“Enter Student Name=”)
- The input taken using input statement is by default is of string type, hence if numerical value is taken as input and if arithmetic operations to be performed on it, then it must be type casted as :
- E.g. num1=int(input(“Enter a number=”))
- Num2=float(input(“Enter a number=”))

e.g

```
import math
```

```
a=int(input("Enter a number="))
```

```
print ("Sq Root=", math.sqrt(a))
```

Output Statements

- The statement in programming code which completes the job to print or write output on any output location.
- Python uses print() lib function to give output at specified output location.
- Python code can use following output destinations :-

1. Std Output Device
2. File

1. Std Output Device – print() by default writes output on std output device as:

```
print(“Hello”)
```

- This Std output can be given in following different forms as:-

i. printing dummy message

```
print(‘Any message’)
```

```
print(“Any message”)
```

ii. printing with escape sequence characters

- as like C, python also allows to use following escape sequence chars

`\n` – new line

`\t` – tab space

```
print(‘Any message \t Any message \n Any Message’)
```

iii. printing defined output or variable output

```
print(VariableName)
```

```
a=10
```

```
print(a)
```

iii. printing combined output – undefined and defined output can be printed, for this comma (,) can be used as separator

```
a=10
```

```
print(‘Value of a=’,a)
```

iv. Printing with format character –As like C lang, python also allows to use format chars such as

`%d` – for integer

`%f` – for float

`%c` – for char

As

```
print(‘ any message %char’ %VariableName)
```

```
A=10
```

```
print(‘int value=%d’ %A)
```

v. printing selected number of digits in real value

```
n=234.6754327
```

```
print('Float value %0.2f' %n)
```

vi. printing without new line

- By default print() gives new line after writing the output on output screen.
- To avoid new line, the “end” parameter can be used as

```
print(‘output’,end=’ ‘) – prints with space
```

```
print(‘output’,end=’\t’) – print with tab space
```

```
print(101,end=’\t’)
```

```
print('Abc')
```

output-

```
101   Abc
```

1.6 Control Structures: loops and decision

- Program code to complete its job get executed in a defined style.

- The style or pattern in which prog code executed is called **control structure or control flow**.

- control structure or control flow in programming can be of following types –

- 1) Sequential flow
- 2) Conditional flow
- 3) Selection flow
- 4) Loop flow

1. Sequential flow – in this flow, line by line program gets executed.

- It is also called as default flow used by any program.

- In this order of program code and order of execution is same.

e.g. program to get input of two number, display original and swapped numbers.

2. Conditional Flow /selection flow/ Decision Making – In this, program code executed on the basis of given condition or expression.

- Following conditional statements can be used in python code-

i) simple if – it is used to implement one-way branching or decision making during program execution.

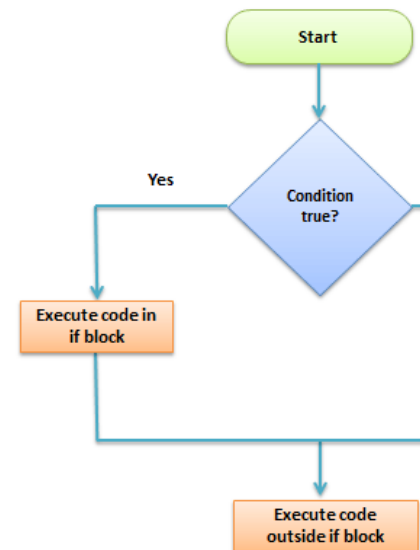
Syntax

```
if (<exp/condition>):
```

```
----
```

```
--- true block----
```

- **enclosing condition in parenthesis is optional.**
- In this, prog uses only true block which is executed when the given condition is true or satisfied.
- After if statement the true block must be indented.



E.g. – program to get input of one number, check and display is it positive number.

```
a=int(input('Enter a Number='))
```

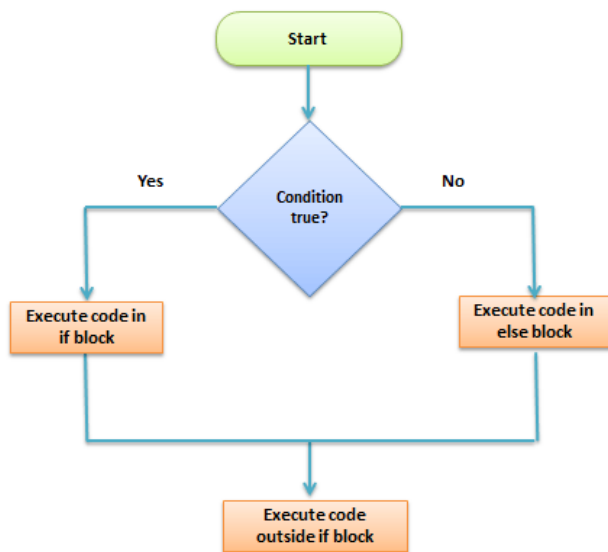
```
if (a>0):
```

```
    print( "Yes")
```

ii) if....else – it is used to implement two-way branching or decision making during program execution.

- It uses two separate block of code, one as true block and other is false block.

- When the condition is satisfied, true block executes, when it is fail, false block executed as:



Syntax

if<condition/exp>:

---true block---

else:

---false block---

- The statements in if or else block must be properly indented.
- **E.g.** – program to get input of one number, check and display is it positive or negative number.

```

a=int(input('Enter a Number='))
if(a>0):
    print('Number is Positive')
else:
    print('Number is Negative')
  
```

iii) if..elif statement –it is used to apply multi-way branching during program execution.

- In this series of conditions are arranged in sequence which are evaluated sequentially.

Syntax:

if<condition1/exp>:

---true block1---

elif<condition2/exp>:

---true block2---

elif<condition3/exp>:

---true block3---

else:

---else / false block---

- Program always evaluates condition 1, when it is satisfied, its true block executed else it go to next condition and so on.
- When none of the condition is true, it executes else block.

e.g. program to get input of one number, check and display is it positive,negative or neutral number.

```

a=int(input("Enter a number"))
if(a>0):
    print("+Ve")
elif (a<0):
    print("-Ve")
else:
    print("Neutral")
  
```

iv) Nested if

- In existing if block other if block enclosed it is called as nested if.
- in simple words, if...with in ...if is also called as Nested if.
- In this, there will be one outer or parent condition and one inner condition.
- First outer condition will be tested, and when this condition is true/satisfied then only it will go for inner condition.
- Thus it helps to define blocks of inter-dependent conditions.

Syntax:

```

if(<Condition1>): #outer condition
    -----
    -----
    if(<Condition1>):#inner condition
        -----
        else: #inner else block
            -----
else:#outer else block
  
```

- When outer condition is true, control enters in it's block, then it will check inner condition.
- When inner condition is true, it executes it's true block otherwise executes inner else block.
- In case, outer condition is fail, it will execute outer else block.

e.g.

prog to get input of one number, check is it Positive or Negative, and if it is positive check is it ODD or EVEN.

```
a=int(input("Enter a number="))
if(a>0):
    print("+Ve")
    if(a % 2 ==0):
        print("EVEN")
    else:
        print("ODD")
else:
    print("-VE")
```

v) using pass statement

- In some cases, if statement to be kept empty, but as per syntax rules, it can not be kept.
- When if statement to be defined with no content or block, it will generate error.
- To avoid this, python supports use of "pass" statement.
- "pass" statement can be used either with if statement or with else statement.
- whenever it is used, that block is skipped for execution.

syntax:

```
if(Condition):
    pass
else:
    --- else block---
```

e.g.1

```
a=int(input("Enter a number="))
if(a>0):
    pass
else:
    print("Value is Negative")
```

OR

```
if(Condition):
    ---true block---
else:
    pass
```

e.g.1

```
a=int(input("Enter a number="))
if(a>0):
    print("+ve")
else:
    pass
```

*** switch ...case replacement**

- python does not support case statement as like in C,C++ or java like languages.
- Instead, python supports alternate technique to implement switch-case.
- In python, switcher{} block can be used to implement switch case.
- In switcher block labeled block to be defined. This label can be int, char, string or float value as:

```
switcher={
    label1:---statement---,
    label2:---statement---,
    .
    .
    .
}
return switcher.get(<exp>,"msg")
```

- In single switcher block, label of different data types can be used..
- return statement is used to return invalid choice.
- To use switcher following steps are needed:
 - define a function with argument
 - define switcher block
- from interpreter call above function with parameter.

e.g.

```
def f1(ch):
    switcher={
        1:'This is case1',
        1.5:'This is case2',
        'a':'This is case3',
        "xyz":'This is case4'
    }
    return switcher.get(ch,'Invalid Choice')
```

- To call above code use following statement

```
>>>f1(1.5)
```

output

This is case2

3. Looping or Iteration Flow

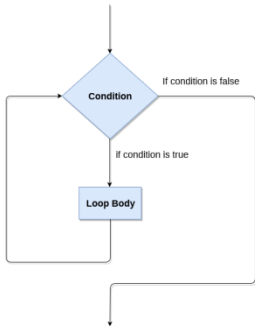
- Looping is the programming technique by which block of code can be repeatedly executed for fixed number of time.
- python supports following looping statements.
 - 1) while statement
 - 2) for statement

1) While statement – This loop statement is used to design general loop codes.

- The Python while loop allows a part of the code to be executed until the given condition returns false.

- **It is also known as a pre-tested loop.**

- with while statement, expression or condition is given which will be tested repeatedly.



- The syntax is given below.

While <exp>:

---- loop body---

e.g. program to display Hello message 5 times

i=1

while i<=5:

 print('Hello')

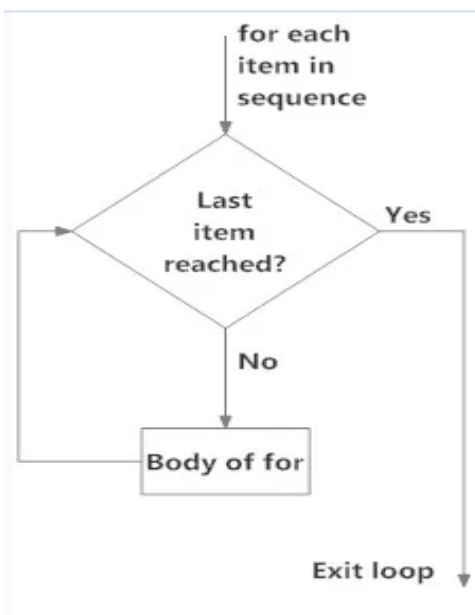
 i=i+1

2) for statement –

- for statement in python is used to repeat execution of given code block.

- As oppose to other languages like C, C++, etc it is used to iterate over a sequence (list, tuple, string) or other iterable objects.

- Iterating over a sequence is called traversal.



- **for loop statement can be used to in following different forms**

i) using range() – this function provides list of fixed range values to execute the loop as:

for <var> in range([start],stop,[step]):

---- loop body---

--

- range() functions uses following different parameters:

i) start – it is optional parameter and it is used as initializing value.

- If not given, by default zero is considered.

ii) stop – it is terminating value which decide when loop to be stopped. Loop executed till stop-1 value.

iii) step – it decides the incrementing value of loop. It can be either positive or negative value.

- Forward order loop – Positive value.

- Reverse Order loop - negative value

- It is optional parameter, if not given, by default it is incremented by 1.

e.g.1

for i in range(3):

 print(i)

output -

0

1

2

e.g.3

for i in range(1,10,2):

 print(i)

output -

1

3

5

7

9

e.g.2

for i in range(1,3):

 print(i)

output -

1

2

e.g.4- Rev for loop

for i in range(5,0,-1):

 print(i)

output -

5

4

3

2

1

e.g.5 for with dict

```
dict={101:'Abc',99.5:'Xyz','aaa':200}
for k in dict.keys():
    print('Data Item at key',k,'=',dict[k])
```

output-

Data Item at key 101 = Abc
Data Item at key 99.5 = Xyz
Data Item at key aaa = 200

e.g.6 for with string

```
s='Hello'
for i in s:
    print(i)
```

output -

H
e
l
l
o

* Loop control statements

- To control the current executing loop, python as like other programming languages C,C++,Java, etc, supports following loop control keywords of statements-

1) continue 2) break

1) continue – this statement is used to skip the current loop iteration and control is jumped to next iteration.

- It is used in those cases when some iterations in loop to be skipped.

- it can be used with conditional statements as:

```
for <exp>:
    [--- some code----]
    if <exp>:
        continue
```

e.g.1

prog to show 1-10 series except 4 & 7 values

```
for i in range(1,11,1):
    if i==4 or i==7:
        continue
    print(i,end=' ')
```

e.g.2 prog to show all small and capital alphabets using single loop

```
for i in range(65,122,1):
    if i>=91 and i<=96:
        continue
    print(chr(i),end=' ')
```

2) break – this loop control statement is used to exit the from loop execution based on given condition.

```
for <exp>:
    [--- some code----]
```

```
if <exp>:
    break
```

e.g.1- prog to repeatedly get input of int numbers below 100 , store in list, as soon as value entered is >=100, loop should stop and display all the values.

```
lst=[]
while True:
    n=int(input("Enter a value="))
    if(n<100):
        print('Ok')
        lst.append(n)
    else:
        break
print(lst)
```

e.g- Prog to find given number is Prime or Non-Prime

Unit – II String Handling, Classes, Modules and Package

2.1 Strings, String operations and Slicing

- As like other languages, python also supports string type data.
- String is collection of character enclosed in single, double or triple quotes.
- python doesn't supports char type, but the single char is also stored and processed as string.
- string can be created in following different forms:

```
s='Abc'
```

```
s="Abc"
```

- Triple quoted string is to create multiline string value as

```
s="'''This is python language which is portable,
extensible and dynamic language'''.
```

e.g.

```
s1='A'
```

```
s2="Computer Language"
```

```
s3="1.Python is Free – Python can be implemented
without purchase of any software or there is no
licensing issues.
```

```
2. Portable – python code can be developed on any
hardware or software platform and can be easily
transferred to any other hardware or software platform.
```

```
3. Extensible"
```

```
print("String1=",s1)
```

```
print("String2=",s2)
```

```
print("String3=",s3)
```

- String in memory is stored as char array with starting index as zero.

e.g.

```
s='Hello'
```

```
01234 → index
```

*String operations

- Python allows to perform different operations on string using following built-in methods:-

1) capitalize() – it will convert only the first character of string to capital & return as new string (original string not changed)

```
s='Abc Xyz'
```

```
print("New String=",s.capitalize())
```

```
print("Original String=",s)
```

Output-

```
New String= Abc xyz
```

```
Original String= Abc Xyz
```

2) title() – it converts first character of each word to capital and returns a new string as -

```
s='abc xyz'
```

```
print("New String=",s.title())
```

```
print("Original String=",s)
```

Output-

```
New String= Abc Xyz
```

```
Original String= abc xyz
```

3) casefold(), lower() – it returns the new string with all characters in small case.

```
s='ABC XYZ'
```

```
#print("New String=",s.casefold())
```

```
print("New String=",s.lower())
```

Output-

```
New String= abc xyz
```

4) upper() – it returns the new string with all characters in upper/capital case.

```
s='abc xyz'
```

```
print("New String=",s.upper())
```

Output-

```
New String= ABC XYZ
```

-the **lower()** function returns the new string with all characters in lower/small case.

```
s='ABC'
```

```
print("New String=",s.lower())
```

Output-

```
New String= abc xyz
```

5) swapcase() – it returns a new string by swapping case of each alphabet in given string, small will be converted to capital and capital to small.

```
s='Abc Xyz'
```

```
print('Swap Case String=',s.swapcase())
```

Output-

```
Swap Case String= aBC xYZ
```

6) isupper() – it returns True if all the characters in string are in upper case else False.

```
s='ABC XYZ'
```

```
print("String is in Upper case?=",s.isupper())
```

Output-

```
String is in Upper case?= True
```

7) islower() – it returns True if all the characters in string are in lower case else False.

```
s='ABC XYZ'
```

```
print("String is in Upper case?=",s.islower())
```

Output-

String is in Upper case?= False

8) find() – searches the given string in parent string, if present returns index position else returns -1

```
s='ABC XYZ'
print("Find result=",s.find('Y'))
print("Find result=",s.find('T'))
```

output

Find result= 5
Find result= -1

9) index() – searches the given string in parent string, if present returns index position **else generates error** -

ValueError: substring not found

```
s='ABC XYZ'
print("Find result=",s.find('Y'))
print("Find result=",s.find('T'))
```

output

index result= 5

ValueError: substring not found

10) isalnum – it returns True, only if string contains either alphabets, digits or both else returns false.

```
s='bcyz'
print('String ',s,' is Alphanumeric=',s.isalnum())
s='9876'
print('String ',s,' is Alphanumeric=',s.isalnum())
s='bc yz'
print('String ',s,' is Alphanumeric=',s.isalnum())
s='bc$yz'
print('String ',s,' is Alphanumeric=',s.isalnum())
```

Output-

String bcyz is Alphanumeric= True
String 9876 is Alphanumeric= True
String bc yz is Alphanumeric= False
String bc\$yz is Alphanumeric= False

11) isalpha() - it returns True, only if string contains alphabets, else returns false.

e.g.

```
s='bcyz'
print(s,' is Alphabetical=',s.isalpha())
s='9876'
print(s,' is Alphabetical=',s.isalpha())
s='bc yz'
```

```
print(s,' is Alphabetical=',s.isalpha())
s='bc$yz'
print(s,' is Alphabetical=',s.isalpha())
s='TTy79'
print(s,' is Alphabetical=',s.isalpha())
```

output

bcyz is Alphabetical= True
9876 is Alphabetical= False
bc yz is Alphabetical= False
bc\$yz is Alphabetical= False
TTy79 is Alphabetical= False

12) isdigit() - it returns True, only if string contains digits or numbers, else returns false.

e.g.

```
s='bcyz'
print(s,' having only digits=',s.isdigit())
s='9876'
print(s,' having only digits=',s.isdigit())
s='bc yz'
print(s,' having only digits=',s.isdigit())
s='bc$yz'
print(s,' having only digits=',s.isdigit())
s='TTy79'
print(s,' having only digits=',s.isdigit())
```

output

bcyz having only digits= False
9876 having only digits= **True**
bc yz having only digits= False
bc\$yz having only digits= False
TTy79 having only digits= False

❖ **String Slicing**

- Python string supports string slicing operation which is to break down string in small parts to make sub-string.

- This slicing is done between given start to end position.

- For this, python supports slicing operator [] which uses following parameters:

[start:stop:step]

- Start - Select chars from given start index
- stop – select chars upto stop-1 index
- step – optional, decides increment step, by default 1.

which can be used in following forms-

i) string[start:] – by this, from given start index till last char, substring is formed.

e.g.
s='ABCDE'
print('Substring=',s[2:])

output-
('Substring=', 'CDE')

ii) string[:end] – by this substring is formed by extracting chars from 0th index till given index-1 position.

e.g.
s='ABCDE'
print('Substring=',s[:2])

output-
('Substring=', 'AB')

iii) string[start:end] – by this substring is formed by extracting chars from given start index till given end index-1 position.

e.g.
s='ABCDE'
print('Substring=',s[1:4])

output-
('Substring=', 'BCD')

iv) string[start:end:step] – by this substring is formed by extracting chars from given start index till given end index-1 position with incrementing chars.

e.g.
s='Maharashtra'
print('Substring=',s[2:7:2])

output-
('Substring=', 'hrs')

v) string[::-step] – by this reverse string generated with step value as -1.

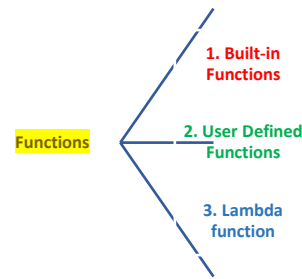
e.g.
s='Maharashtra'
print('Substring=',s[::-1])

output-
('Substring=', 'hsrahtraM')

2.2 Defining and calling functions, passing

arguments to functions

- one of the fundamental criteria of any programming language is to support the concept of function.
- function is self-contained block of instructions to complete a special task.
- Functions in python are broadly classified as:



- 1) Built-in functions – these are the functions which are already defined, compiled and stored in respective library.e.g sqrt() – from math library.
- 2) User Defined Functions – These are functions created for a specific user application.
 - UDF can be created using def keyword, a function can be defined with or without parameter and also with or without return as:
def FunName([parm]):
--- block---
[return]
 - **When return keyword not used, by default it returns none object.**
 - Once a function is created, it can be called any time as:
funcName([arg])

e.g.1 function with-out parameter

```
def fun1():
    print("Function Without argument")
    - To call above functions, on python interpreter following statement can be used:
    >>>fun1()
```

e.g.2 function with one parameter

```
def fun2(a):
    print("Value in a=",a)
    - To call above functions, on python interpreter following statement can be used:
    >>>fun2(10)
```

e.g.3 function with multiple parameters

```
def fun1(a,b,c):
    print("Value in a=",a)
    print("Value in b=",b)
    print("Value in c=",c)
```

To call above functions, on python interpreter following statement can be used:

```
>>> fun1(99,'Abc',96.7)
('Value in a=', 99)
('Value in b=', 'Abc')
('Value in c=', 96.7)
```

e.g.4 function with single return

```
def fun1(a):
    return a*a
```

To call above functions, on python interpreter following statement can be used:

```
>>> n=fun1(10)
>>> print ('Square=',n)
```

e.g.5 function with multiple return

```
def fun1(a):
    return (a,a*a)
```

To call above functions, on python interpreter following statement can be used:

```
>>> num,sq=fun1(7)
>>> print ('Number=',num,'Square=',sq)
```

Output

```
Number=7 Square=49
```

- Alternatively above function can be called from interpreter using single receiving variable as:

```
n=fun1(9)
print(n)
output-
(9, 81)
```

e.g.6 function with default argument

- In this type, function is defined with parameter and a default value is assigned.
- Such function can be called by passing argument or without argument.
- If argument is passed, it is stored in parameter and processed.
- If argument not passed, the default value is used for processing.

e.g.

```
def fun1(a,b=2):
    print("Addition=",a+b)
```

e.g.7 function with variable number arguments:-

- In some cases, function to be defined to receive arguments, but number of arguments to be passed are not fixed.

- In such case, a function is defined with single argument but preceded by * (asterisk) symbol as:

```
def funName(*ArgName):
    ---- block---
```

- such argument get created as collection in memory and stores multiple values which can be accessed using any loop statement.

e.g.

```
def fun1(*args):
    for i in args:
        print("value=",i)
```

- Above function can be called by passing multiple arguments of same or different types:

```
>>> fun1(99,33,44)
('value=', 99)
```

```
('value=', 33)
('value=', 44)
```

```
>>> fun1(23,'abc',8.9,True)
('value=', 23)
('value=', 'abc')
('value=', 8.9)
('value=', True)
```

3) **Lambda function** – generally a function is defined by giving proper name, but python also allows to create a function without name.

- Such function without name or anonymous function is called lambda function.

- A function with-name defined with “def” keyword, but a lambda function is defined with “**lambda**” keyword.

- This function to define uses 3 parts

- i) lambda keyword
- ii) single or multiple arguments
- iii) expression – single line executable statement.

- a lambda function can return only 1 value, but it does not uses “return” keyword.

syntax

```
lambda arg1,arg2... : expression
```

e.g.1

```
>>> x=lambda a:a+5
>>> print("Expression value=",a(5))
```

Output

```
Expression value= 10
```

e.g.2

```
>>> x=lambda a,b:a*b
>>> print("Multiplication=",x(3,5))
```

Output

```
Multiplication= 15
```

2.3 Defining Classes

- As python is OOP language hence it supports all the features of OOP concepts.

- Class is one such most fundamental concept of OOP supported by every OOP lang.

- Class is single security unit which allows to combine data as well as function member.

- In python program, class can be created using “**class**” keyword as:

```
class <ClassName>:
    [data member]
```

[function member]

- A class can enclose data as well as function members, but both are always not compulsory.
- while giving the class body, following rules must to be followed:

i) when a function member to be defined, it must have at least one parameter i.e. self.

ii) when a function member needs to access, class data member, this self keyword must be used as:

self.MemberName

- Once class definition is over, it's member can be accessed using object.

- Object is one runtime entity or instance which represents the class.

- Object is memory block, which reserves space of data and function member.

- Object can be declared as:

ObjName=ClassName()

e.g. ob1=ClassA()

- This object with Member accessing operator (.) is used to access member of class.

Eg.1

class A:

n=0 #initializing class member

def display(self):

self.n=100

print("Value of n=",self.n)

- To call above function,

>>> ob=A()

>>> ob.display()

Output

Value of n=100

e.g. 2 function with argument

class A:

def display(self,n):

print(n)

- To call above function,

>>> ob=A()

>>> ob.display(89)

Output

Value of n=89

- Python class supports following access specifiers:

i) private – when the any member of is declared as private, it is accessible only with-in class.

- To make any member private, it must starts with double underscore as:

__rollno=0

ii) protected – when the any member of class is declared as protected, it is accessible in same class & only from it's derived classes.

- to make any member protected, it must starts with single underscore as:

_rollno=0

iii) public – by default member of class in python is public. If any member is public can be accessible within the class and also outside the class.

- to make any member public, no underscore is used as:

Rollno=0

❖ Constructor & Destructor

- Every class created in OOP language supports two special member functions i.e. Constructor and Destructor.

Constructor – The special member function of class that performs the task of **allocation and initialization** is called constructor.

- It shows following special features in python programming:

i) it is implicitly created as well as called.

ii) constructor in python class can be created using def keyword with name `__init__(self)` as:

```
def __init__(self):  
    ---- code block ----
```

iii) constructor get called whenever object of class is created.

Destructor – The special member function of class that performs the task of **de-initialization** **and de-allocation** is called destructor.

- It performs the task opposite and in reverse as of constructo.

- It shows following special features in python programming:

i) it is implicitly created as well as called.

ii) Destructor in python class can be created using def keyword with name `__del__(self)` as:

```
def __del__(self):  
    ---- code block ----
```

iii) Destructor get called whenever object of class is deleted using '**del**' keyword as:

```
del <objectName>
```

e.g. of constructor & destructor

class A:

```
n=0;
```

```
def __init__(self):
```

```
    self.n=10
```

```
def display(self):
```

```
    print("Value to n initialized by
```

```
constructor=",self.n)
```

```
def __del__(self):
```

```
    print("Destructor Called, Object Deleted")
```

- **To execute above**

```
>>> ob=A()
```

```
>>> ob.display()
```

```
('Value to n initialized by constructor=', 10)
```

23

```
>>> del ob
```

Destructor Called, Object Deleted

2.4 Python and OOP –

- As python is OOP language, hence it supports all the features of OOP and gives all it, benefits.

- To most common features of OOP are:

Inheritance,

polymorphism

❖ Inheritance

- Inheritance is one of the feature of OOP language.

- This OOP technique by which from an existing class a new class is created.

- The existing class is called Base/Parent/Super class and the newly created class called as Derived/Child/Sub class.

-When inheritance is applied, it shows following effects:

i) Copy of data member is transferred from base to derived class &

ii) permission given to child class to call base class function without object.

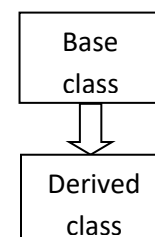
- Inheritance gives biggest advantage of reusability.

- In Python, following types of inheritances are supported:

1) Single 2) Multi-level 3) Multiple

4) Hierarchical 5) Hybrid

1) Single – In this inheritance, only one base class and only one derived class is involved:



Syntax : Class DerivedClassName(BaseClsName):

----- class definition -----

e.g.

class A:

n=0;

def display(self):

self.n=input("Enter any Value=")

print("in Base Class n=",self.n)

class B(A):

def show(self):

self.display()

print("in Derived Class n=",self.n)

===== To Execute=====

>>> ob=B()

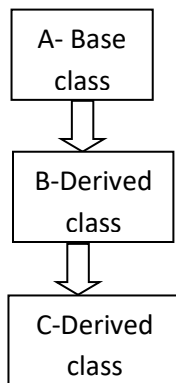
>>> ob.show()

Enter any Value=123

('in Base Class n=', 123)

('in Derived Class n=', 123)

2) Multi-level – In this inheritance, from an existing child/derived class a new class is created:



- Thus in above eg. From A Base class, B derived class is created, and from this B Derived class, another derived class is created i.e. C

Syntax : Class DerivedClassName(BaseClsName):

----- class definition -----

e.g.

class A:

n=0;

24

def display(self):

self.n=input("Enter any Value=")

print("in Base Class A, n=",self.n)

class B(A):

def show(self):

print("in Derived Class B, n=",self.n)

class C(B):

def output(self):

self.display()

self.show()

print("in Derived Class C, n=",self.n)

===== To Execute=====

>>> ob=C()

>>> ob.output()

Enter any Value='Abc'

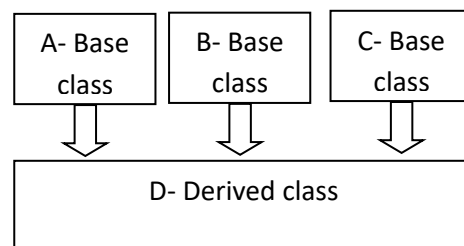
('in Base Class A, n=', 'Abc')

('in Derived Class B, n=', 'Abc')

('in Derived Class C, n=', 'Abc')

3) Multiple/Multipath – In this inheritance, two or more base classes are use to create a child class.

- The child class inherits properties from each base class and also can call member function of each base class.



class A:

n=0

def getIn1(self):

self.n=int(input("Enter Ist Value="))

class B:

m=0

def getIn2(self):

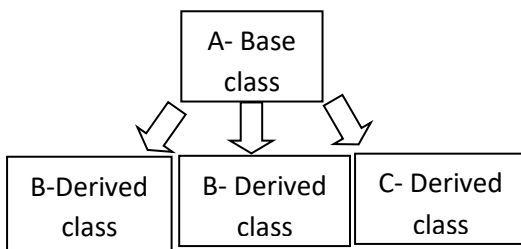

```

self.m=int(input("Enter IInd Value="))
class C(A,B):
    def display(self):
        self.getIn1()
        self.getIn2()
        print("Addition=",self.n+self.m)
===== to execute =====
>>> ob=C()
>>> ob.display()
Enter Ist Value=12
Enter IInd Value=23
('Addition=', 35)

```

4) Hierarchical – In this inheritance, from single base class, multiple derived classes are created.

- It is used to provide common features to all derived classes.
- Such common features are available in base class and inherited by all derived class.



e.g.

```

import math
class A:
    n=0
    def __init__(self):
        self.n=int(input("Enter a Value="))
class B(A):
    def f1(self):
        print("Square=",self.n*self.n)
class C(A):
    def f2(self):
        print("Cube=",self.n*self.n*self.n)

```

25

```

class D(A):
    def f3(self):
        print("Sq Root=",math.sqrt(self.n))
ob1=B()
ob1.f1()
ob2=C()
ob2.f2()
ob3=D()
ob3.f3()
del ob1
del ob2
del ob3

```

=====output=====

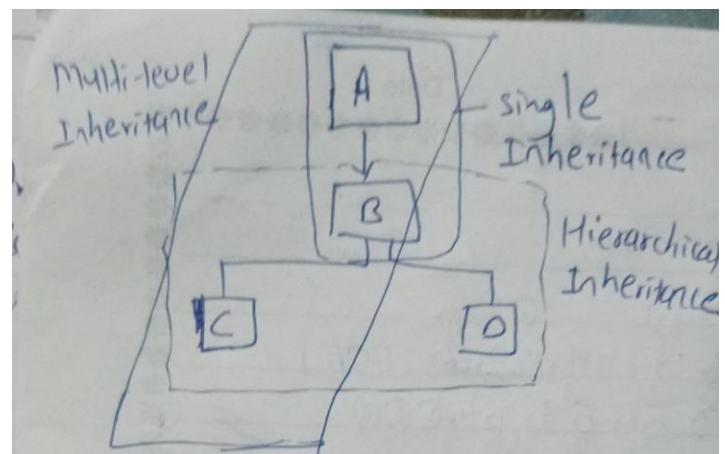
```

Enter a Value=12
('Square=', 144)
Enter a Value=5
('Cube=', 125)
Enter a Value=7
('Sq Root=', 2.6457513110645907)

```

5) Hybrid – In this inheritance, other inheritance types are combinedly used.

- It is used to get combined benefits of other inheritance types:



e.g.

```

import math

```

```

class A:
    n=0
class B(A):
    def __init__(self):
        self.n=int(input("Enter a Value="))
class C(B):
    def f1(self):
        print("Cube=",self.n*self.n*self.n)
class D(B):
    def f2(self):
        print("Sq Root=",math.sqrt(self.n))
ob1=C()
ob1.f1()
ob2=D()
ob2.f2()
del ob1
del ob2

```

=====output=====

Enter a Value=7

('Cube=', 343)

Enter a Value=9

('Sq Root=', 3.0)

❖ Polymorphism

- Polymorphism word is made up of two words i.e. Poly=many, morphs=types or forms.
- This OOP technique is to make a programming entity available in multiple forms.
- The programming entity which is available in multiple form is said to be polymorphic.
- It can be implemented in python language in following ways-

1) Using built-in functions- many library functions of Python language shows polymorphism i.e. same function can be used in different forms.

e.g.1 print() function can be used to show value of int, float, char, string, Boolean, list, tuple, set, dictionary , etc.

function can be to calculate length of different types of values like, string, list, set, dictionary.

```
print("String Len=",len('abcd'))
```

```
lst=[101,'abc',8.9]
```

```
st={100,200,300,'Abc','Xyz'}
```

```
dct={1:100,2:200,3:300}
```

```
print("List Len=",len(lst))
```

```
print("Set Len=",len(st))
```

```
print("Dict Len=",len(dct))
```

2.5 Modules – Python interpreter allows to creates functions, declarations or other executable statements on interpreter prompt(>>>).

- It is easy to create, but these are available for temporary purpose, as soon as the interpreter is closed, it get removed from memory.
- In many cases, a lengthy function is necessary to be repeatedly executed.
- If such function is not stored, then repeatedly it needs to be created and then called, which very

Time consuming

Error prone – chances of generating errors

- To solve this, python supports the concept of module.
- **Module is a file which can contain, classes, declarations, executable statements or functions definitions.**

- Once it is saved, it can be called any number of times.

- Python supports many built-in modules and also allows to create user defined module.

- Commonly used built-in modules in python are

Datetime

Math

❖ **datetime** - Python Datetime module supplies classes to work with date and time.

- These classes provide a number of functions to deal with dates, times and time intervals.
- Date and datetime are an object in Python, so when you manipulate them, you are actually manipulating objects and not string or timestamps.
- Commonly used classes in the datetime module are:

I. **date Class** – object is this class represents year, month and day.

- The current date and all date parts can be extracted.
- Current date can be displayed as:

```
>>> import datetime
>>> print(datetime.date.today())
2022-05-14
```

- Date parts can be extracted by creating date object as:

```
<obName>=datetime.date.today()
```

E.g.

```
from datetime import date
ob=date.today()
print("Year=",ob.year)
print("Month=",ob.month)
print("Day=",ob.day)
```

```
====output=====
```

```
('Year=', 2022)
```

```
('Month=', 5)
```

```
('Day=', 14)
```

II. **time Class** – it represents time values like hours, minutes, seconds and milliseconds
e.g.

```
from datetime import datetime
```

```
now = datetime.now()
```

```
print(now)
```

```
=====output=====
```

```
2022-05-14 07:05:10.453000
```

- to print formatted time string

```
from datetime import datetime
```

```
now = datetime.now()
```

```
current_time = now.strftime("%H-%M-%S")
```

```
print("Current Time =", current_time)
```

```
=====output=====
```

```
('Current Time =', '14-08-33')
```

III. **timedelta Class** – this class represents date and time manipulations by calculating difference between two date and time.

- by using “+” symbol date or time values can be added and “-” symbol used to subtract values to return new date or time value.

e.g from datetime import datetime,timedelta

```
time_obj = datetime.now()
```

```
d1=time_obj.strftime("%d/%m/%Y")
```

```
print("Current Date=",d1)
```

```
d2=time_obj+timedelta(days=2)
```

```
d2=d2.strftime("%d/%m/%Y")
```

```
print("New Date=",d2)
```

```
t1 = time_obj.strftime("%H-%M-%S")
```

```
print("Current Time =", t1)
```

```
t2=time_obj + timedelta(hours=3)
```

```
t2=t2.strftime("%H-%M-%S")
```

```
print("new Time=",t2)
```

```
=====output=====
```

```
('Current Date=', '14/05/2022')
```

```
('New Date=', '16/05/2022')
```

('Current Time =', '14-34-41')

('new Time=', '17-34-41')

❖ **math** – python provides math as built-in module to complete different mathematical task.

- it consist of many built-in methods and constants to complete mathematical operations.

- to use this module & it's methods, it must be imported using import keyword as:

```
import math
```

- Any method of math module can be called as:

```
math.funName([arg])
```

- Following are the few built-in functions of math module:

i) math.ceil() – it takes one float argument and returns it's next nearest value as:

```
import math
```

```
print("Ceil value=",math.ceil(7.6))→8.0
```

```
print("Ceil value=",math.ceil(7.1))→8.0
```

ii) math.fabs() – it returns the absolute value in the form of float number:

```
print("Abs value=",math.fabs(3))→3.0
```

```
print("Abs value=",math.fabs(-3))→3.0
```

iii) math.factorial()- it returns factorial of given number:

```
print(math.factorial(5)) →120
```

iv) math.floor() -it returns nears down floating value by eliminating fractional part

```
math.floor(8.99) →8.0
```

v) math.fmod(x,y) – it returns the remainder by dividing x by y

```
math.fmod(5,3) → 2.0
```

vi) math.fsum()-returns sum of all items from lists, tuple, set etc only if all items are numeric else shows TypeError

```
lst=[2,5,5,10]
```

```
math.fsum(lst) → 17.5
```

vii) math.isqrt() – returns down integer of floating square root value as:

```
print('Only Sq root=',math.sqrt(8))- 2.8284271247
```

```
print('int sq root=',math.isqrt(8)) – 2
```

viii) math.pow(x,y) – return power of x raised to y as:

```
math.pow(3,3) → 27.0
```

ix) math.prod() – returns multiplication value of all items in tuple, list,set etc if all items are numeric.

```
lst=[2,3,5.5]
```

```
print('Prod of All Items=',math.prod(lst))→33.0
```

x) math.trunc()- returns only the int part by truncating fractional part of given float value

```
math.trunc(9.9)→9
```

❖ **User defined module**

- Python also allows to create and use user-defined module in following steps;

→ create required number functions and save

in a file with .py extension.

→ the file name is considered as module name.

→ import the above file/module

→ call the function with module name

e.g.

```
def display(a,b):
```

```
    print("Addition=",a+b)
```

```
    print("Subtn=",a-b)
```

```
    print("Div=",a/b)
```

```
    print("Mult=",a*b)
```

→ save above file as arith.py

→ import this module as

```
import arith
```

→ call module function as

```
arith.display(9,8)
```

2.6 Packages – Python allows to organize all the related classes, function etc in same module like Date class, time class, datetime class is in datetime package, all mathematical functions are in math module.

- similarly, it is also possible that, their might be group of related modules to be organized properly.

- Group of related modules organized in same location/folder is called package.

- Suppose you have developed a very large application that includes many modules.

- As the number of modules grows, it becomes difficult to keep track of them all if they are dumped into one location.

- This is particularly so if they have similar names or functionality. You might wish for a means of grouping and organizing them.

- **Packages** allow for a hierarchical structuring of the module namespace using **dot notation**.

- In the same way that **modules** help avoid collisions between global variable names, **packages** help avoid collisions between module names.

- Python supports both types of packages –

i) Built-in packages – these are the packages which come by default with every python installation.

- all these are stored in lib folder after installation e.g. collections, html, http, sqlite3, etc.

ii) user defined packages – the package created for a specific user application.

- To create and use user-defined package, following steps to considered

i) create a folder in current python installation folder

e.g. MyPack1

ii) in this package, create & modules containing class, functions, etc

e.g. mod1

```
def display():
```

```
    print("This is user Def Package")
```

- The above module to saved and compiled

iii) create, save and compile empty `__init__.py` file

iv) to call/access above package import the package in one of the following forms

a) using import – by this the entire library can be loaded and become accessible e.g.

```
import MyPack
```

```
import datetime
```

b) using from...import statement – by this only the requires modules can be imported as

```
from MyPack import Billing
```

e.g

```
>>> from MyPack1 import mod1
```

```
>>> mod1.display()
```

Unit – III - Exception Handling and Collections

3.1 Exception in python

3.2 Exception roles

- In Python programming, exception are used for following different purposes, hence exception plays important role in following cases:

1. Error Handling – Python interpreter raises exception whenever their occurs runtime error.

- In such case following are the possibilities:-

i) Ignore the error, if error ignored, pythons default exception handler will be called, it will show the corresponding message and terminates program execution.

ii) handle the error using try and catch statements, by this successfully error will be handled and program will complete it's normal execution.

2. Event Notification – Many times entry of invalid data or invalid operation needs to explicitly checked, which is very time consuming. Hence exception can be used to notify such invalid condition.

3. Termination actions – Some times, a software uses such special operation code which must be executed in any case before closing the program called as **House Keeping operations** like Closing DB Connection , Closing DB, Closing File, etc.

- the try/finally block when used, ensures such action is taken before termination of program in any case, whether exception occurs or not.

4. Unusual control flows – Other languages like C or C++ supports jumping statement i.e. goto, by which program control can be transferred from one part to other.

But as like java, python also don't support goto, instead exception with **raise keyword** can be used for same purpose.

3.3 Exception Handling

3.4 Collections in Python – List, Tuples, Dictionaries, Sets

[this part is already discussed in Chap-1 under data type heading]

Unit – IV - GUI Programming and Database Connectivity Using Python

4.1 Graphical User Interfaces

- The working environment provided by any software code or OS is called interface by which user interacts with hardware & other components.

- Based on their working environment, interfaces are classified as:

1) CLI – Command Line Interface - In this, to complete each task, correct command in proper form and syntax to be entered.

-Adv – Such interface is easy to create.

Minimum Hardware requirements.

Dis-Adv – Difficult to use, understand, error prone and time consuming.

2) GUI – Graphical User Interface – In this, to complete each task, corresponding commands or actions are presented on screen.

- Such interface also called, point-n-click or Point-n-select interface.

- It uses different graphical components like, menus, controls (Label, Button, Textfield, radio button, etc), image, icons, etc.

Adv – More easy to understand and use, less chances of errors and time saving.

Dis-Adv – More complex to design for programmer.

Requires extra hardware.

- Python provides different library modules to design GUI such as:

1) **Tkinter** is a standard **GUI** (graphical user interface) package.

Tkinter is Python's default GUI module and also the most common way that is used for GUI programming in Python.

Note that Tkinter is a set of wrappers that implement the Tk widgets as Python classes.

2) wxPython:

This is basically an open-source, cross-platform GUI toolkit that is written in C++.

Also an alternative to Tkinter.

3) JPython:

JPython is a Python platform for Java that is providing Python scripts seamless access to **Java class Libraries** for the local machine.

4.2 Using the tkinter Module

Tkinter is Python's default GUI module and also the most common way that is used for GUI programming in Python.

- Tkinter stands for ToolKit interface. Toolkit as like in java is built-in library.

-This toolkit in Python provides modules to design GUI using python.

- this module in older version of python interpreter i.e. before 3.0 is available as TKinter and now it is renamed to tkinter.

- The GUI once created provides a parent or container window in which other controls can be added.

- In Python, the GUI controls are called **widgets** & different such widgets can be used are :

Text, Button, info Dialog Boxes, Radiobutton, Checkbutton , etc

- Steps to create GUI using tkinter module

i) import tkinter module as

from tkinter import tkinter

or

from tkinter import *

ii) create object of `Tk()` by calling its constructor `Tk()` to create window as:

`objName=Tk()`

e.g.

```
import tkinter
```

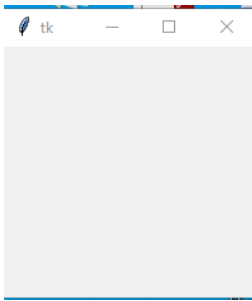
```
top=tkinter.Tk()
```

- Above statement will create and show a window with

Default size

Default bgcolor

Default title as tk



- To this GUI, some basic properties can be applied as:

i) Setting title – `<obj>.title("any title")`

ii) setting window size - `<ob>.geometry("int width x int height")`

iii) setting background color – background color can be applied in one of the following ways:

a) using `configure` function with `bg` attribute-
`<ob>.configure(bg='ColorName')`

```
ob.configure(bg='Red')
```

b) directly using `bg` property -

```
<ob>['bg']='ColorName'
```

```
ob['bg']='Red'
```

or

```
<ob>['bg']='#ColorCode'
```

```
ob['bg']='#127744'
```

e.g.

```
from tkinter import *
```

```
gui=Tk()
```

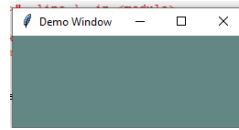
```
gui.title("Demo Window")
```

```
gui.geometry("100x100")
```

```
#gui.configure(bg=#120,67,89)
```

```
#gui['bg']='red'
```

```
gui['bg']='#628784'
```



4.3 Creating Label – Label is one GUI widget used as text control.

- It is used to show static as well as dynamic text as well as image on gui.

- Steps to create and use Label widget:

i) create and initialize label object as:

`<objName>=Label()` – it will create an empty label with default properties.

Or

`<objName>=Label(ParenWindowObj,text="any text",bg='ColorName',fg='ColorName', font="Font attributes")`

ii) To show/ load label in window, use one of the following method

a) `pack()` – this method adds label object at default location on GUI.

b) `place(Co(x)=int, row(y)=int)`

e.g.

```
from tkinter import *
```

```
gui=Tk()
```

```
gui.title("Label Demo")
```

```
gui.geometry("100x200")
```

```
gui['bg']='#209970'
```

```
lb1=Label(gui,text="Demo1",bg='red',fg='yellow',font='Script 40 bold underline')
```

```
#lb1.place(x=00,y=50)
```

```
lb1.pack()
```




- Label widget also allows to handle key as well as mouse events.

- To handle label event use following steps:

i) create a U.D. function with at least one arg as:

```
def MouseClick(arg):
```

----- code----

ii) link the event to label object using bind() as:

```
<labObj>.bind("<Event>", FnName)
```

- python provides many built-in event names like

Button

Key , etc

e.g.

```
from tkinter import *
```

```
def mouseClick(arg):
```

```
    print('Mouse Clicked')
```

```
gui=Tk()
```

```
gui.title("Label Demo")
```

```
gui.geometry("200x200")
```

```
gui['bg']='#209970'
```

```
lb1=Label(gui,text="Demo1",bg='red',fg='yellow',font  
='Script 40 bold underline')
```

```
#lb1.place(x=00,y=50)
```

```
lb1.pack()
```

```
lb1.bind("<Button>",mouseClick)
```



- Label widget can also be used to show image in gui.

- To add image in Python GUI, following supporting components are needed:

- o Canvas – class
- o PhotoImage – function

- The steps to add image in GUI using Label are:

- i) Create & initialize Canvas object.
- ii) Add canvas object in GUI.
- iii) Create and initialize image object using PhotoImage().
- iv) Call create_image() of Canvas class to display the image.

e.g.

for following e.g. mm.png file is stored in current folder of python program.

```
from tkinter import *
```

```
gui=Tk()
```

```
gui.title("Label Demo")
```

```
gui.geometry("200x200")
```

```
canvas=Canvas(gui,width=300,height=300)
```

```
canvas.pack()
```

```
img=PhotoImage(file="mm.png")
```

```
canvas.create_image(20,20,image=img)
```

Text – This python gui widget from tkinter library is used to create dynamic text entry control so that user can provide text input.

- To create and use Text widget following steps are required:

i) Create and initialize Text object as:

```
<objName>=Text(ParentWindow,[height=int,width=int,bg='colname',fg='colname',font='properties'])
```

- By default, text widget works in multi-line mode, to make it single line, height property must be set to 1.

ii) add object in GUI

-
- Text class for different manipulations, supports following functions:-

i) insert(Position,"text") – by this method any text can be inserted in text widget using INSERT as it's position constant.

```
t1.insert(INSERT,"Abc Xyz")
```

ii) **get(start,end)** –by this method, current value from text control can be read as:

x=t1.get(1.0,"end") – by this from first to last char all contents are read in x variable.

iii) **delete(start,end)** – by this method, current value of text control can be deleted as:

```
t1.delete(1.0,"end")
```

e.g.

```
from tkinter import *
```

```
def fn():
```

```
    x=t1.get(1.0,"end")
```

```
    t2.delete(1.0,"end")
```

```
    t2.insert(INSERT,x)
```

```
gui=Tk()
```

```
gui.geometry("300x300")
```

```
t1=Text(gui,height=1)
```

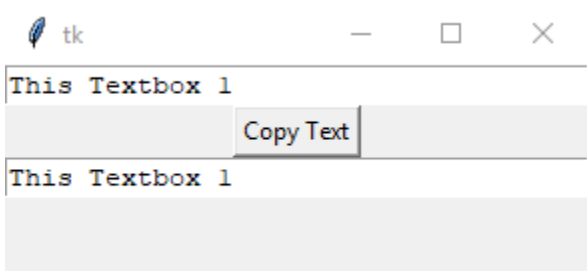
```
t2=Text(gui,height=1)
```

```
btn=Button(gui,text="Copy Text",command=fn)
```

```
t1.pack()
```

```
btn.pack()
```

```
t2.pack()
```



Button, -

- This widget of python tkinter library is used to create command button or push button control to be displayed on GUI.

- This button helps to take necessary action to user as per the given execution flow.

- Button can be created and used in following steps:

i) create button object as:

```
BtnObj=Button(ParentWindow,text='text',[fg='color name',bg='colorname',activebackground='color name',command=functionName])
```

- There are many optional parameter used like:

fg- to specify foreground color

bg- to specify background color

activebackground – helps to set the color which is displayed whenever button is pressed.

Command – is used to link the button event with a function which get called whenever button is clicked.

ii) Add the button object in GUI using pack() or place() method.

e.g.

```
from tkinter import *
```

```
from datetime import *
```

```
def fn():
```

```
    dt=datetime.now()
```

```
    t1.insert(INSERT,str(dt))
```

```
gui=Tk()
```

```
gui.geometry("300x300")
```

```
lb=Label(gui,text="Current Date & Time=")
```

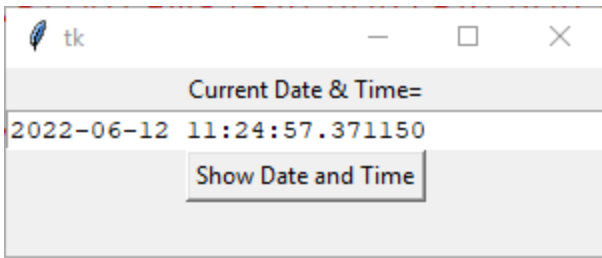
```
t1=Text(gui,height=1)
```

```
btn=Button(gui,text="Show Date and Time",command=fn)
```

```
lb.pack()
```

```
t1.pack()
```

```
btn.pack()
```



info Dialog Boxes – one of the important feature of and standard GUI is, it must communicate/interact with it's user.

- For this purpose, GUI supports dialog box controls.
- dialog box is a small fixed sized window which is used to get input from user, give output to user or both.
- Dialog boxes are child window which are always displayed in parent window.
- tkinter library of Python supports many built dialog boxes which can be activated by calling their respective functions.
- The older version of python interpreter support tkMessageBox module and the newer version supports messagebox module.
- These modules supports following built-in functions to show dialog boxes.

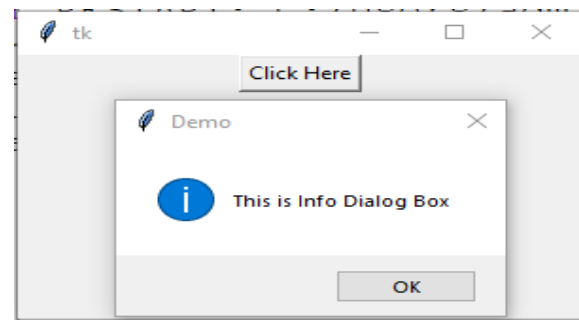
i) messagebox.showinfo(title,"messageText") – this dialog box is used to show information msg to user, it comes with default “OK” button and information icon.

- When the Ok button is pressed, it returns “ok”.

e.g.

```
from tkinter import *
from tkinter import messagebox
def fn():
    x=messagebox.showinfo("Demo","This is Info
Dialog Box")
    print(x)
gui=Tk()
gui.geometry("300x300")
btn=Button(gui,text="Click Here",command=fn)
```

btn.pack()

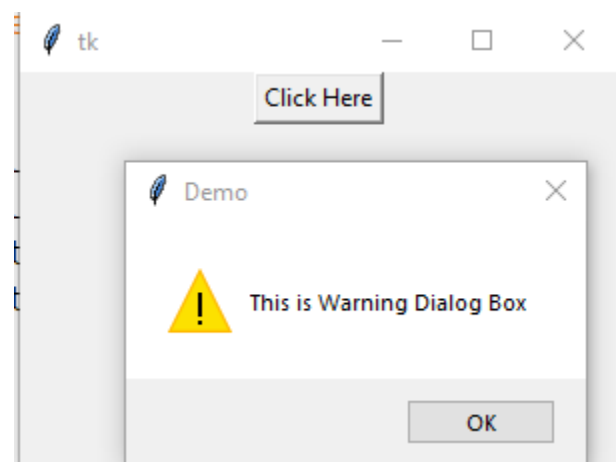


ii) messagebox.showwarning(title,"messageText") – this dialog box is used to show warning msg to user, it comes with default “OK” button and yellow colored warning icon.

- When the Ok button is pressed, it returns “ok”.

e.g.

```
from tkinter import *
from tkinter import messagebox
def fn():
    x=messagebox.showwarning("Demo","This is Info
Dialog Box")
    print(x)
gui=Tk()
gui.geometry("300x300")
btn=Button(gui,text="Click Here",command=fn)
btn.pack()
```



iii) messagebox.showerror(title,"messageText") – this dialog box is used to show error msg to user, it

comes with default “OK” button and red colored error icon.

- when the Ok button is clicked, it returns “ok” as string.

e.g.

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
def fn():
```

```
    x=messagebox.showerror("Demo","This is Info  
Dialog Box")
```

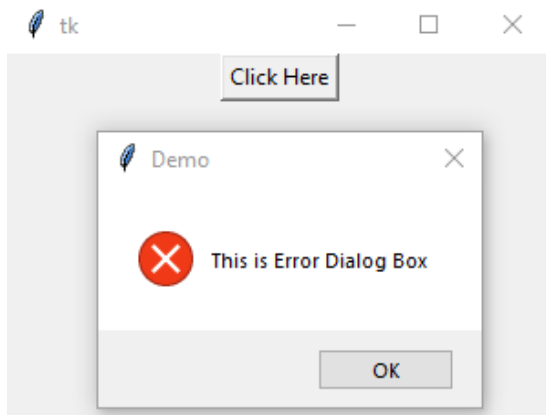
```
    print(x)
```

```
gui=Tk()
```

```
gui.geometry("300x300")
```

```
btn=Button(gui,text="Click Here",command=fn)
```

```
btn.pack()
```



iv) messagebox.askquestion(title,”messageText”) – this dialog box is used to show question msg to user, it comes with default “Yes” and “No” button and red question mark icon.

- when the Yes button is clicked, it returns “yes” as string, when the No button is clicked, it returns “no” as string,

e.g.

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
def fn():
```

```
    x=messagebox.askquestion("Demo","This is  
Question Dialog Box")
```

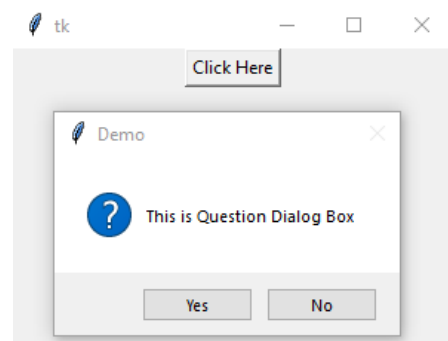
```
    print(x)
```

```
gui=Tk()
```

```
gui.geometry("300x200")
```

```
btn=Button(gui,text="Click Here",command=fn)
```

```
btn.pack()
```



v) messagebox.askokcancel(title,”messageText”) – this dialog box is used to show Ok Cancel Dialog box to user, it comes with default “Ok” and “Cancel” button and red question mark icon.

- when the Ok button is clicked, it returns “True” as, when the Cancel button is clicked, it returns “False”.

E,g,

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
def fn():
```

```
    x=messagebox.askokcancel("Demo","This is Ok  
Cancel Dialog Box")
```

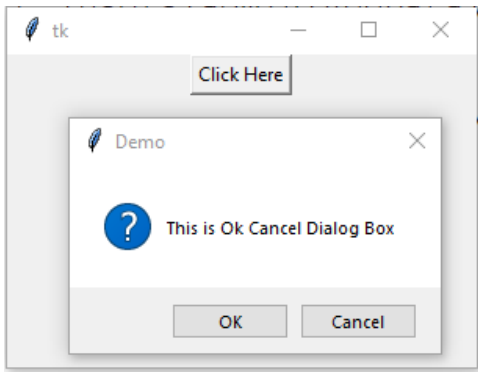
```
    print(x)
```

```
gui=Tk()
```

```
gui.geometry("300x200")
```

```
btn=Button(gui,text="Click Here",command=fn)
```

```
btn.pack()
```



vi) `messagebox.askyesno(title,"messageText")` – this dialog box is used to show Yes No dialog box to user, it comes with default “Ok” and “Cancel” button and red question mark icon.

- when the Yes button is clicked, it returns “True” as, when the No button is clicked, it returns “False”.

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
def fn():
```

```
    x=messagebox.askyesno("Demo","This is Yes No Dialog Box")
```

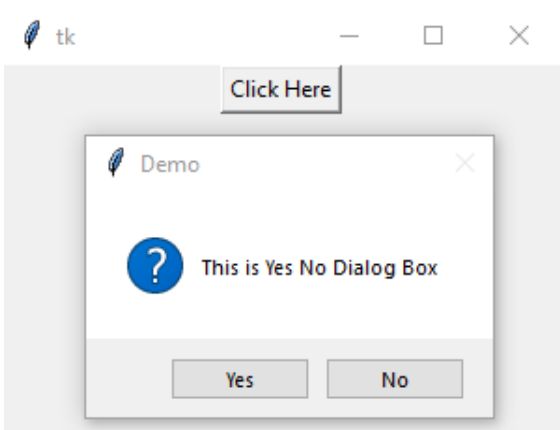
```
    print(x)
```

```
gui=Tk()
```

```
gui.geometry("300x200")
```

```
btn=Button(gui,text="Click Here",command=fn)
```

```
btn.pack()
```



vii)

`messagebox.askretrycancel(title,"messageText")` – this dialog box is used to show retry cancel dialog box

to user, it comes with default “Retry” and “Cancel” button and red question mark icon.

- when the Retry button is clicked, it returns “True” as, when the Cancel button is clicked, it returns “False”.

e.g.

```
from tkinter import *
```

```
from tkinter import messagebox
```

```
def fn():
```

```
    x=messagebox.askretrycancel("Demo","This is Retry Cancel Dialog Box")
```

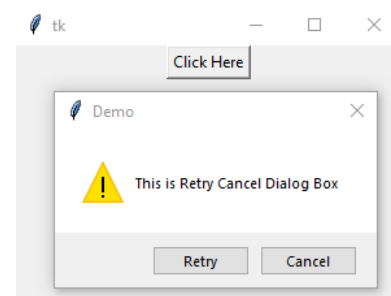
```
    print(x)
```

```
gui=Tk()
```

```
gui.geometry("300x200")
```

```
btn=Button(gui,text="Click Here",command=fn)
```

```
btn.pack()
```



Radiobutton,

- One of the important use of any GUI is as data collection form.

- For data collection, two possible techniques are used:

i) direct entry – When uncertain data to be collected from user like Student Name, employee name, etc, this technique is used.

It is easy to use but time consuming and also error prone.

ii) Selection entry – in this user is provided with selectable option to collect certain or known data like Blood group. Gender, HSC boards of Maharashtra State, etc.

- By this data collection is easy, time saving and also error-free.
- For this, GUI supports selectable controls.
- This widget of tkinter library helps to provide set of selectable options to user.
- **It allows single selection from one group.**
- It appears as circle shaped widget on screen and shows dot as selection mode.
- It works only in toggle mode i.e. either ON/True or OFF/False mode.
- Steps to create and use radio button widget in GUI

- ➔ Create one int variable using IntVar() function.
- ➔ Create and initialize Radiobutton object with above variable associated as:

```
Obj=Radiobutton(ParentWindow, text="any text",
Variable=VarName, value=value,
command=FunctionName)
```

- ➔ Use value property and assign value for execution.
- ➔ To handle the radio button event, define function and bind using command attribute.
- ➔ Add radiobutton object in gui using pack() or place() method.

e.g.

```
from tkinter import *
```

```
def fn():
```

```
    lb.config(text="Option Selected="+str(v1.get()))
```

```
gui=Tk()
```

```
gui.geometry("300x200")
```

```
lb=Label(gui)
```

```
v1=IntVar()
```

```
rb1=Radiobutton(gui,text="Option 1",variable=v1,
value=1,command=fn)
```

```
rb2=Radiobutton(gui,text="Option 2",variable=v1,
value=2,command=fn)
```

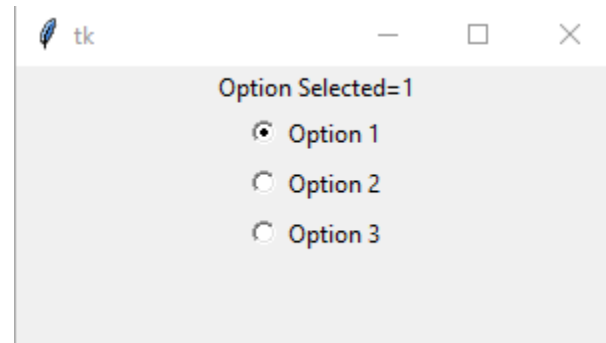
```
rb3=Radiobutton(gui,text="Option 3",variable=v1,
value=3,command=fn)
```

```
lb.pack()
```

```
rb1.pack()
```

```
rb2.pack()
```

```
rb3.pack()
```



Checkbutton

- One of the important use of any GUI is as data collection form.
- For data collection, two possible techniques are used:

i) direct entry – When uncertain data to be collected from user like Student Name, employee name, etc, this technique is used.

It is easy to use but time consuming and also error prone.

ii) Selection entry – in this user is provided with selectable option to collect certain or known data like Blood group. Gender, HSC boards of Maharashtra State, etc.

- By this data collection is easy, time saving and also error-free.
- For this, GUI supports selectable controls.
- This widget of tkinter library helps to provide set of selectable options to user.
- It allows **multiple selection** from one group.

- It appears as square shaped widget on screen and shows ✓ mark as selection mode.
- It works only in toggle mode i.e. either ON/True or OFF/False mode.
- Steps to create and use radio button widget in GUI

- ➔ Create one int variable using IntVar() function.
- ➔ Create and initialize Checkbutton object with above variable associated as:
Obj=Checkbutton(ParentWindow, text="any text", Variable=VarName, command=FunctionName)
- ➔ Use value property and assign value for execution.
- ➔ To handle the checkbutton event, define function and bind using command attribute.

Add checkbutton object in gui using pack() or place() method.

e.g.

```
from tkinter import *
```

```
def fn():
```

```
    lb.config(text="Status:
```

```
    CB1="+str(v1.get())+"\tCB2="+str(v2.get()))
```

```
gui=Tk()
```

```
gui.geometry("300x200")
```

```
lb=Label(gui,font="bold")
```

```
v1=IntVar()
```

```
v2=IntVar()
```

```
cb1=Checkbutton(gui,text="Option
```

```
1",variable=v1,command=fn)
```

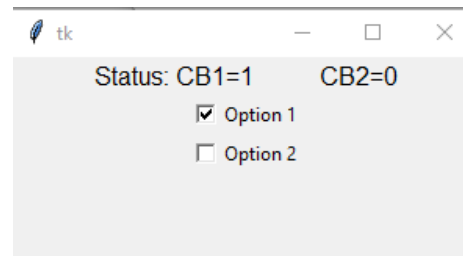
```
cb2=Checkbutton(gui,text="Option
```

```
2",variable=v2,command=fn)
```

```
lb.pack()
```

```
cb1.pack()
```

```
cb2.pack()
```



4.4 **Getting Input**

- One of the important feature of and standard GUI is, it must communicate/interact with it's user.

- For this purpose, GUI supports dialog box controls.

- dialog box is a small fixed sized window which is used to get input from user, give output to user or both.

- tkinter library of Python supports **Entry widget** which is used to create input control in gui.

- entry widget show one text entry control on gui in which use can enter the required input.

- to read the value of entry object, it support get() method.

- input widget can be created in following steps:

➔ create Canvas Object

➔ add canvas object in GUI.

➔ Create Label object and add in Canvas using create_window()

➔ create Entry object and add in Canvas using create_window()

➔ create button object and add in canvas using create_window()

➔ define a function to handle button event and link with button using command property.

e.g.

```
from tkinter import *
```

```
def fn():
```

```
    x=entryObj.get()
```

```
    lb2.config(text="Hello..." +str(x))
```

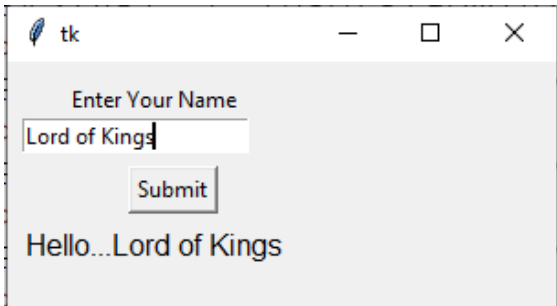
```
gui=Tk()
```

```

gui.geometry("300x200")
canObj=Canvas(gui)
lb1=Label(gui,text="Enter Your Name")
canObj.create_window(80,20,window=lb1)
entryObj=Entry(gui)
canObj.create_window(70,40,window=entryObj)
btn=Button(gui,text="Submit",command=fn)
canObj.create_window(90,70,window=btn)

lb2=Label(gui,font="bold")
canObj.create_window(80,100,window=lb2)
canObj.pack()

```



4.5 **Importing MySQL for Python**

- Python programming language supports many features, one such important feature is, Database programming.
- Python code can be connected with any backend database and can perform read as well as write operations.
- MySQL is a relational database management system (RDBMS) developed by Oracle based on structured query language (SQL).
- To implement DB programming in Python with MYSQL following are necessary components:

i) MySQL server – ensure MySQL server 5.5 or onwards version is installed on system.

ii) mysql connector for Python – Python code to be connected with MySQL needs a special component or driver i.e. mysql connector.

- mostly, by default, to check whether it is installed or not, use following command on python interpreter

```
>>> import mysql.connector
```

- if it is not installed, above command shows following error

```
ModuleNotFoundError: No module named 'mysql'
```

- to solve this use following steps:

→ Goto command prompt

→ Goto script location of python installation folder, goto scripts folder and run pip (Package Installer for Python) command as:

```
C:\Users\admin\AppData\Local\Programs\Python\Python
37-32\Scripts>pip install mysql-connector-python
```

- if the system is having active internet connection, the mysql.connector will be installed & it will show the status as: Successfully installed.

4.6 **Connecting with a database**

4.7 **Forming a query in MySQL**

4.8 **Passing a query to MySQL**

[for above 3 headings use following common explanation]

- Python programming language supports many features, one such important feature is, Database programming.

- Python code can be connected with any backend database and can perform read as well as write operations.

- MySQL is a relational database management system (RDBMS) developed by Oracle based on structured query language (SQL).

- To implement DB programming in Python with MYSQL following are necessary components:

i) MySQL server

ii) mysql connector for Python

- If above two components are ready, python code can be connected with MySQL Database in following steps

→ import mysql connector as:

```
import mysql.connector
```

→ create and initialize connection object using connect() method of mysql.connector as:

```
ConObj=mysql.connector.connect(host,user,password,da
tabase)
```

- this connect method uses 4 parameters as:

i) host – value of this parameter use to specify physical location of MySql server or the computer on which it is installed.

- if it is on same machine, “localhost” value can be used also IP address can be specified for other machine in network like host=”121.1.1.1”

- ii) **user** – Value of this parameter use to specify user id required to connect with mysql server, by default it is “root”
- iii) **password** - Value of this parameter use to specify password which is set during MySQL server installation.
- iv) **database** - Value of this parameter use to specify which particular database to be connected as MySQL server might having many databases.

Forming Query & Passing query

- Once the connection is established with MySQL database, Python code can perform read, write or both operations.
- to complete any such operation, sql query to be composed & executed.
- for this, MySQL, as like Oracle uses concept of **Cursor** which is temporary memory location used internally for SQL operations by server.
- Hence, in python code also Cursor object to be created using connection object as:
CurObj=con.cursor()
- with this cursor object, execute() method can be called to which SQL statement must be passed as parameter like:

CurObj.execute(“sql stmt”)

- Cursor also supports another useful method i.e. fetchall() by which record set (all records) can be extracted in python variable as:

RecordSetObj=CurObj.fetchall()

e.g.

- Consider on MySQL server MyDB1 database is available with following table and records

StudInfo

StudID	StudName	Course
1001	David	BCA
1002	Peter	BCA
1003	John	BBA

e.g.1 python code to read a record

```
import mysql.connector
con=mysql.connector.connect(host="localhost",user="root",password="abc#123",database="MyDB1")
cur=con.cursor() # creating cursor object
cur.execute(“select * from StudInfo”) # executing sql stmt
rst=cur.fetchall() # extracting all records in record set
print(rst) # it will print first record
```

output

```
[(1001,'David','BCA')]
```

e.g.2 python code to read all records

```
import mysql.connector
con=mysql.connector.connect(host="localhost",user="root",password="abc#123",database="MyDB1")
cur=con.cursor() # creating cursor object
cur.execute(“select * from StudInfo”) # executing sql stmt
rst=cur.fetchall() # extracting all records in record set
print(“Stud ID\t\tStud Name\t\tCourse”)
print(“=====”)
for x in rst:
    print(x[0]+”\t”+x[1]+”\t”+x[2])
```

output

Stud ID	Stud Name	Course
=====		
1001	'David',	'BCA'
1002	'Peter',	'BCA'
1003	'John',	'BBA'